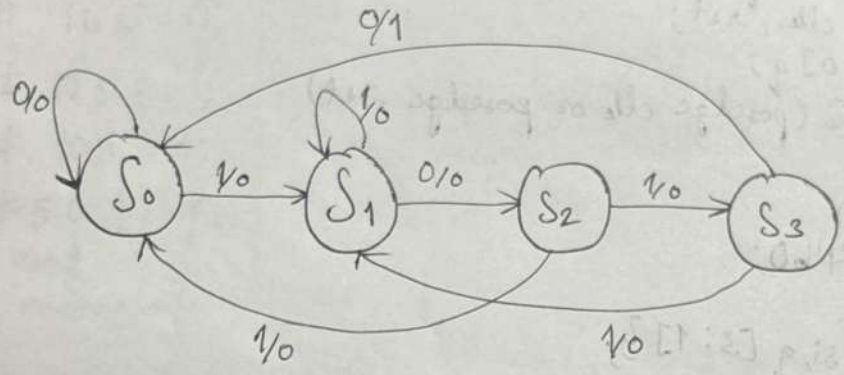
Assessment -6

1) verilog code for mealy and moore in modelism

THEORY

Mealy machine's output depends on both the current state & the current input
Output changes immediately when the input changes.
Generally, it requires fewer states than Moore.

DJAGRAM



VERILOG CODE FOR MEALY FSM

```
module seq_mealy(y,i,clk,rst);
output y;
input i,clk,rst;
reg y;
reg [1:0] cs,ns;
parameter S0 = 2'b00;
parameter S1 = 2'b01;
parameter S2 = 2'b10;
parameter S3 = 2'b11;
always @(cs or i)
 begin
 y = 1'b0
 case (cs)
 S0 : begin
 if(i)
 ns = S1;
```

```
else
ns = s0;
end
s1:begin
if(i)
ns=s1;
else
ns=s2;
end
s2:begin
if(i)
ns=s3;
 else
ns=s0;
end
s3:begin
if(i)
ns=s1;
 else begin
 ns=s2;
 y=1'b1;
 end
 end
 endcase
  end
```

MEALY FSM TESTBENCH

```
module seq_mealy_test;
wire y;
reg i,clk,rst;
seq_mealy f1(y,i,clk,rst);
always
#5 clk=~clk;
initial
begin
i=1; clk=0; rst=1;
#10 rst=0;
#10 i=0;
#10 i=1;
#10 i=0;
#10 i=1;
#10 i=0;
#50 $stop;
end
endmodule
```

```
// current state logic - sequential logic
always @(posedge clk or posedge rst)
begin
if (rst)
cs <= s0;
else
cs <= ns;
end
endmodule
```

PROCEDURE

1. Open model SIM and create new project
2. Add verilog source files for both mealy & moore sequence detectors
3. Write testbench code
4. Simulate after compiling all files to check for errors.
5. Get output & graph.
6. Compare the timing of outputs between mealy & moore models.

CONCLUSION

Mealy machine responded faster since its output depends on both state & input, while the Moore machine gave more stable output based on only the state.

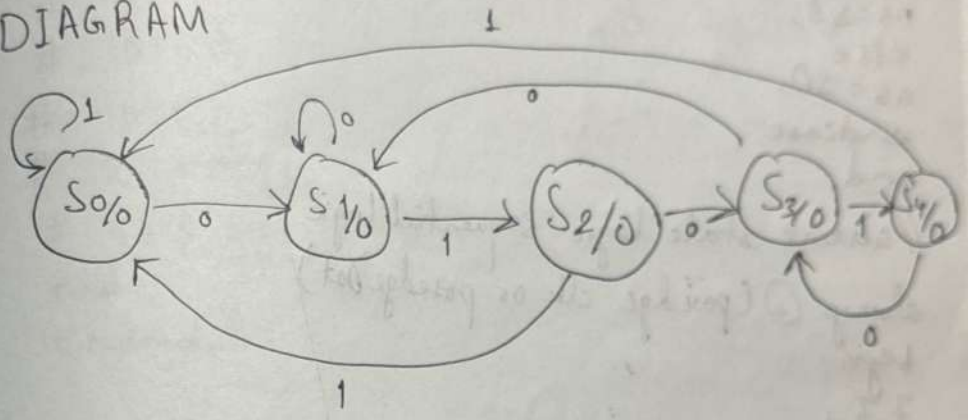
2) Design & implement 0101 sequence detector using Moore Machine

TOPIC : Moore machine

THEORY:

A moore state machine is a finite state machine (FSM) where the output is determined solely by its current state.
That means output changes only when the state changes usually on the next clock edge, making it more stable & predictable.

DIAGRAM



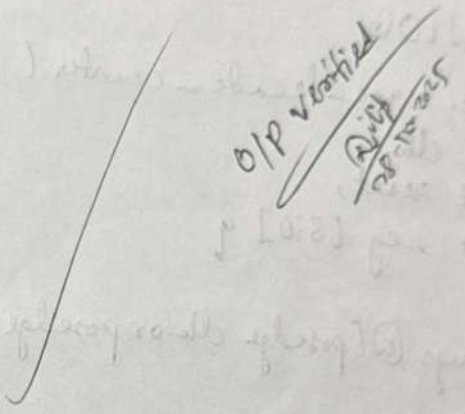
VERILOG

```
module seq_moore(y,i,clk,rst);
output y;
input i,clk,rst;
reg y;
reg [2:0] cs, ns;
parameter S0 = 3'b000;
parameter S1 = 3'b001;
parameter S2 = 3'b010;
parameter S3 = 3'b011;
parameter S4 = 3'b100;
always @(cs or i)
begin
case (cs)
S0 : if (i)
ns = S1;
else
```

```
ns=s0;
s1: if(i)
ns=s1;
else
ns=s2;
s2: if(i)
ns=s3;
else
ns=s0;
s3: if(i)
ns=s1;
else
ns=s4;
s4:if(i)
ns=s3;
else
ns=s0;
endcase
end
//current state logic-sequential logic
always @(posedge clk or posedge rst)
begin
//
if(rst)
cs<=s0;
else
cs<=ns;
end
//output logic -combinational
logic
always @(cs)
begin
if(cs==s4)
y=1'b1;
else
y=1'b0;
end
endmodule
```

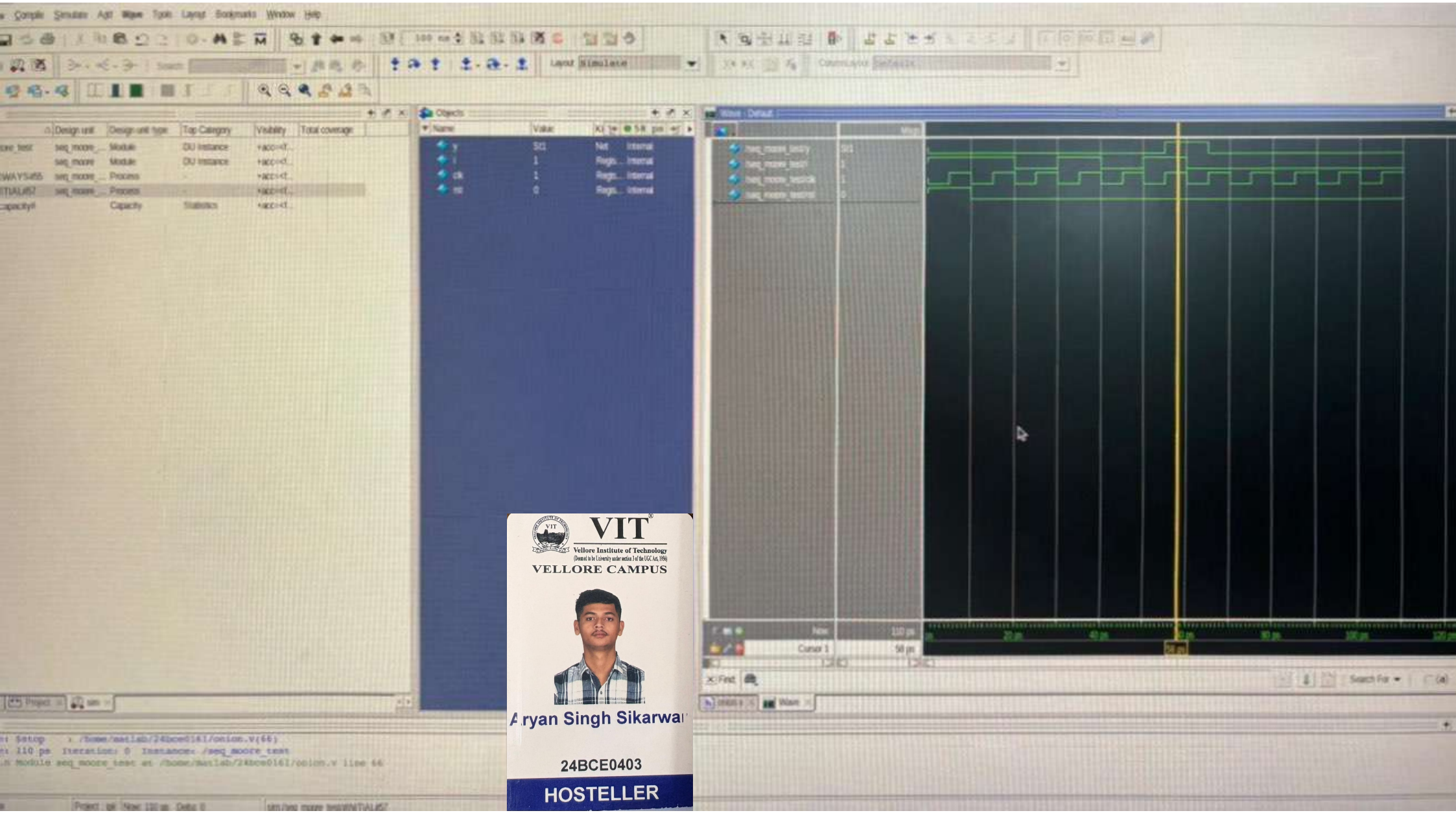
TESTBENCH

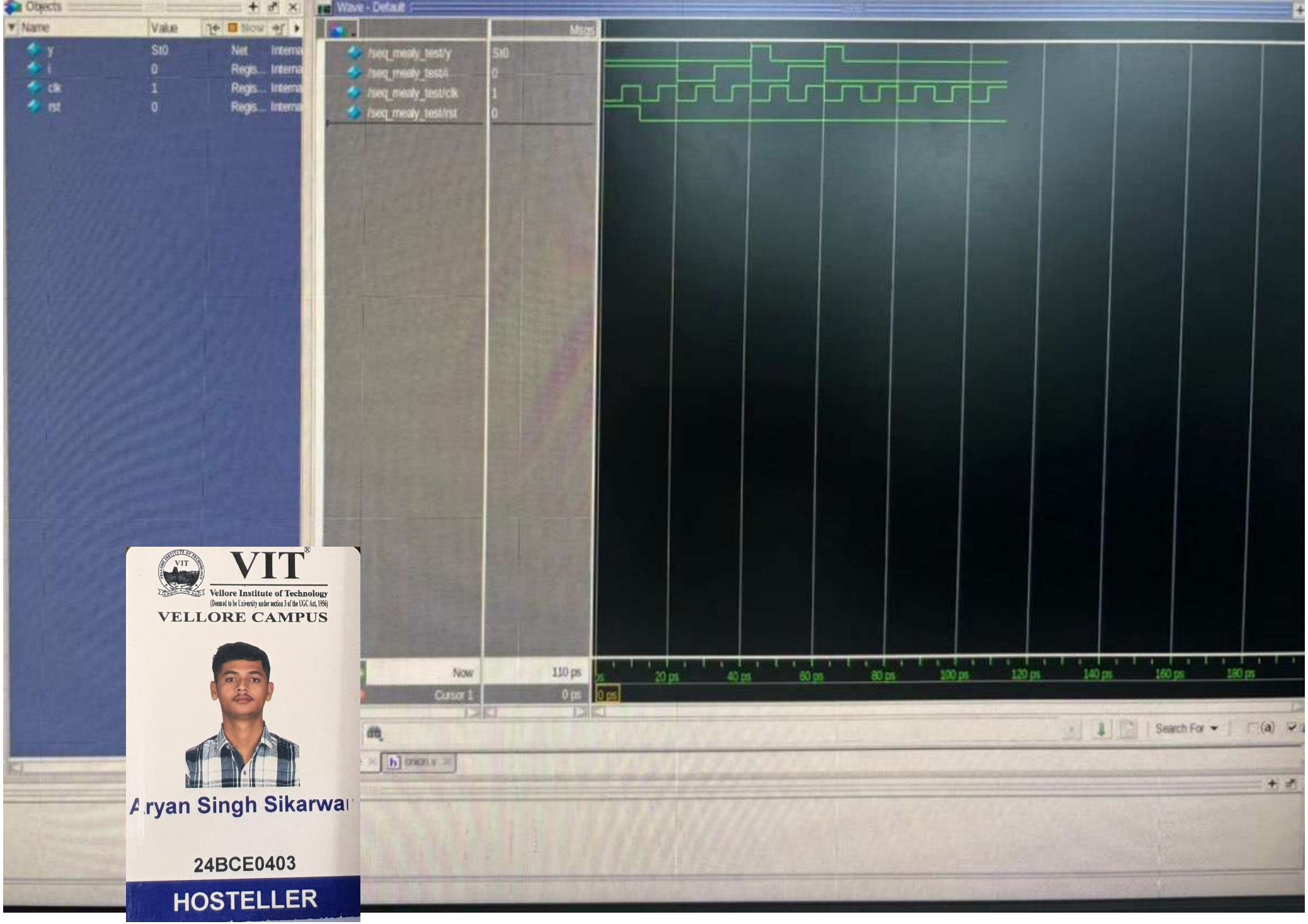
```
module seq_moore_test;
wire y;
reg i, clk, rst;
seq_moore f1 (y, i, clk, rst);
always
# 5 clk = ~clk;
initial
begin
i = 1; clk = 0; rst = 1;
# 10 rst = 0;
# 10 i = 0;
# 10 i = 1;
# 10 i = 0;
# 10 i = 1;
# 10 i = 0;
# 50 $stop;
end
endmodule
```



CONCLUSION

Its output depends only on the current state, making it stable & easier to design compared to the mealy machine.





2. Write verilog code for asynchronous decade counter.

Theory

An asynchronous counter (also called a ripple counter) is a type of counter where the flip-flops are not clocked simultaneously, instead one flipflop triggers the next.

VERILOG

```
module async_decade_counter (
input clk,
input reset,
output reg [3:0] q
);
always @(posedge clk or posedge reset)
begin
 if (reset)
 q <= 4'b0000;
 else if (q == 4'b1001)
   q <= 4'b0000;
 else
   q <= q+1;
 end
 endmodule
module async_decade_counter_tb;
  reg clk;
  reg reset;
  wire [3:0] q;
 async_decade_counter uut(
    .clk (clk);
    .reset ( reset),
    .q (q)
   );
  initial begin
  clk = 0;
  forever #5 clk = ~clk;
  end
```

```
initial begin
 reset =1;
# 10;
 reset  =0;
 # 200;
 reset  =1;
 # 10;
 reset  = 0;
 # 100;
 $stop;
 end
 initial begin
 $monitor("Time = %0t | Reset = %b | Count = %b (%0d)",
 $time, reset, q, q);
 end
 endmodule
```

CONCLUSION

The counter correctly counts from 0 to 9 and resets to 0, verifying the proper operation of a mod-10 asynchronous counter